

Programming in Java

(BCAC 0012)

Module-2

Faculty Name – Mr. Shivank Chauhan

Module 2 Section 3

Packages and Interfaces

Packages And Interfaces	Defining Packages, Importing Packages, User Defined packages and predefined packages
	Introduction to Interfaces.

Syllabus

- 1. Defining Packages**
- 2. Importing Packages**
- 3. User Defined packages and predefined packages**
4. Introduction to Interfaces.

Packages in Java

- A java package is a group of similar types of classes, interfaces and sub-packages.
- A Package can be defined as a grouping of related types (classes, interfaces, enumerations and annotations) providing access protection and namespace management.

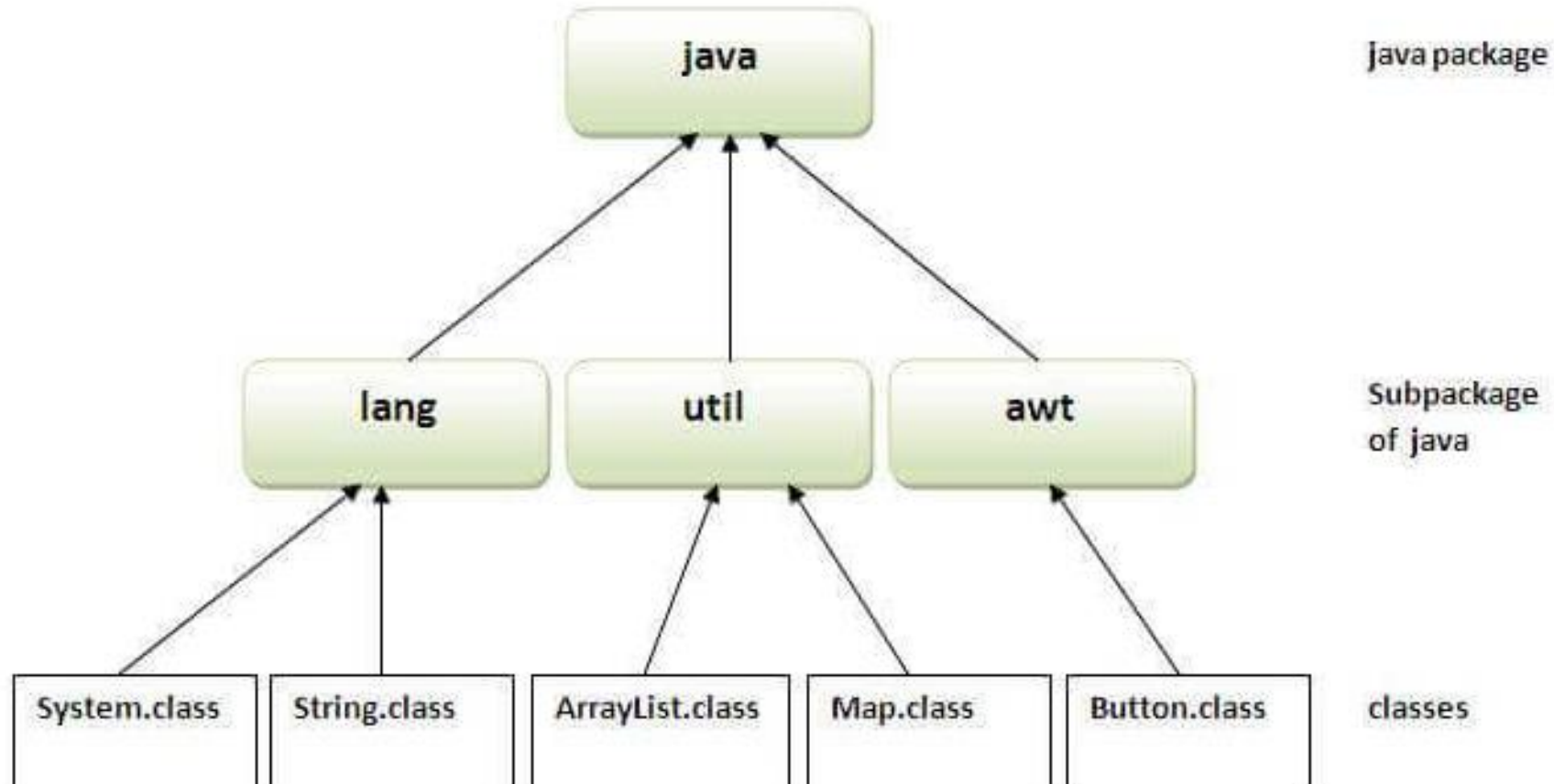
Packages in Java

- Package in java can be categorized in two forms, built-in package and user-defined package.
- There are many built-in packages such as lang, awt, javax, swing, net, io, util, sql etc.
- We can also create our user defined packages.

Advantage of Java Package

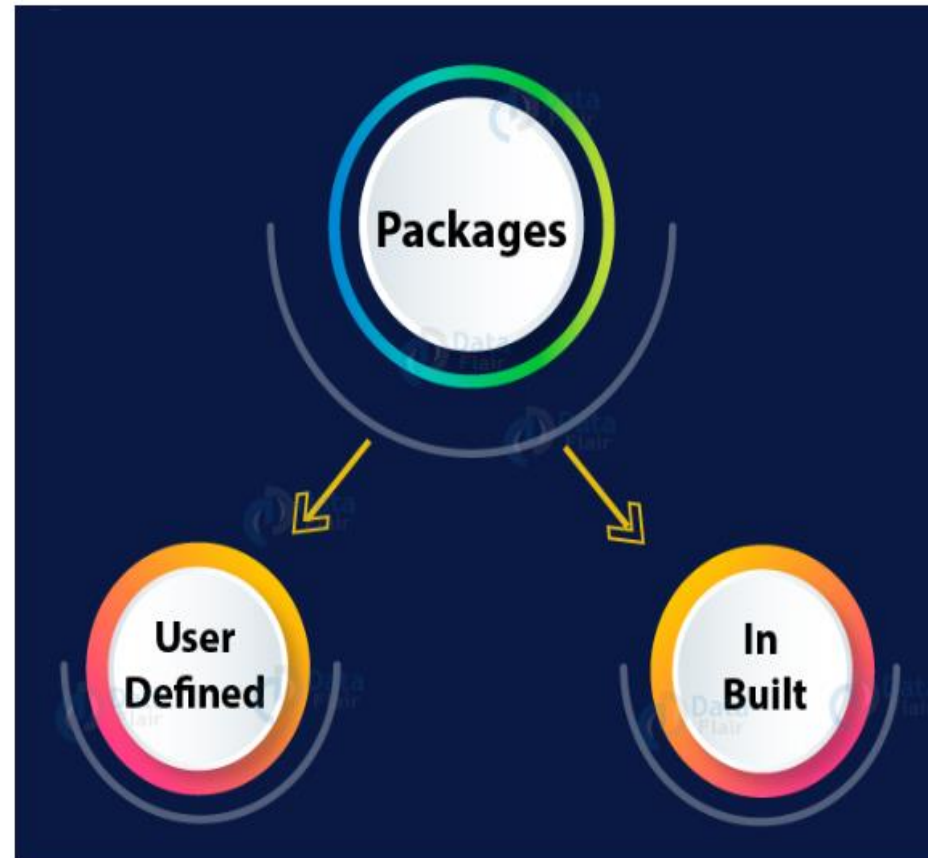
- 1) Java package is used to categorize the classes and interfaces so that they can be easily maintained.
- 2) Java package provides access protection.
- 3) Java package removes naming collision.

Packages in Java



There are two types of Packages in Java-

1. User Defined packages
2. predefined packages (In-Built packages)



Predefined packages or Built in packages

- Built-in is a part of Java API.
- These packages come with JDK and we can directly import and use these packages.
- In every java program the java.lang package is already imported and no need to import it.
- We have to import all other packages as and when needed.

Predefined packages or Built in packages

- Java offers a variety of built in packages.
- The six major packages offered by Java API are :
 1. **lang** – Automatically imported and it contains language support classes.
 2. **io** – Contains classes for input and output operations.
 3. **util** – Contains utility classes for implementing data structures.
 4. **applet** – This package contains classes that create applets.
 5. **awt** – Contain classes that implement compounds for GUI.
 6. **net** – This package contains classes that support networking operations.

User Defined Packages : Defining Packages

- In Java, the user creates these packages according to their requirements.
- While creating a package, you should choose a name for the package and include a **package** statement along with that name at the top.

User Defined Packages : Defining Packages

- Example

```
package mypack;  
public class Simple  
{  
    public static void main(String args[])  
    {  
        System.out.println("Welcome to package");  
    }  
}
```

User Defined Packages : Defining Packages

- The package statement should be the first line in the source file.
- There can be only one package statement in each source file.
- If a package statement is not used then the class, interfaces, enumerations, and annotation types will be placed in the **current default package**.

User Defined Packages : Defining Packages

- How to compile and run the Simple.java program

// Simple.java

```
package mypack;
```

```
public class Simple
```

```
{
```

```
    public static void main(String args[]){
```

```
        System.out.println("Welcome to package");
```

```
    }
```

```
}
```

User Defined Packages : Defining Packages

- **To Compile:** `javac -d . Simple.java`
- **It will create a directory mypack and store Simple.class file in mypack directory**
- **To Run:** `java mypack.Simple`
- The `-d` switch specifies the destination where to put the generated class file.
- If you want to keep the package within the same directory, you will use. (dot) in between `-d` and `Simple.java` in the command of compilation.

User Defined Packages : Defining Packages

- So after creating Simple.java in jdk/bin directory use following command to run it.
- To Compile: `javac -d . Simple.java`
- To Run: `java mypack.Simple`

C:\WINDOWS\system32\cmd.exe

```
D:\jdk\bin>javac -d . Simple.java
```

```
D:\jdk\bin>java mypack.Simple  
Welcome to package
```

```
D:\jdk\bin>_
```


Importing Packages

- We can import classes from one package to another package to use them.
 1. `import package.classname; //` to import a single class from a package
 2. `or import package.*; //` to import all classes from that package

Importing Packages

Example

Let we have a class A in package **pack**

```
package pack;  
public class A  
{  
    public void msg()  
    {  
        System.out.println("Hello");  
    }  
}
```

Importing Packages

- **To Compile** : `javac -d . A.java`
- It will create a folder/package with name pack and store A.class file in that folder
- OR
- **To Compile** : `javac A.java`
- Create a folder with name 'pack' in 'BIN' folder
- Copy the A.class file from BIN to BIN\pack\ folder

Importing Packages

// Importing class from a package

//save by B.java

```
import pack.*;  
class B{  
    public static void main(String args[])  
    {  
        A obj = new A();  
        obj.msg();  
    }  
}
```

Importing Packages

- **To Compile** : javac B.java
- **To Run** : java B
- Output:
- Hello

C:\WINDOWS\system32\cmd.exe

```
D:\jdk\bin>javac -d . A.java
```

```
D:\jdk\bin>javac -d . B.java
```

```
D:\jdk\bin>java mypack.B  
Hello
```

```
D:\jdk\bin>
```

How to access package from another package?

- There are three ways to access the package from outside the package.
 1. `import package.*;`
 2. `import package.classname;`
 3. fully qualified name.

1. Using `packagename.*`

- If you use `packagename.*` then all the classes and interfaces of this package will be accessible but not subpackages.
- The `import` keyword is used to import the classes and interface of package.

Example

```
//A.java
package pack;
public class A
{
    public void msg()
    {
        System.out.println("Hello");
    }
}

//B.java
package mypack;
import pack.*;
class B
{
    public static void main(String a[])
    {
        A obj = new A();
        obj.msg();
    }
}
```

Output: Hello

2. Using packagename.classname

- If you import packagename.classname then only declared class of this package will be accessible.

Example

```
//A.java
package pack;
public class A
{
    public void msg()
    {
        System.out.println("Hello");
    }
}

//B.java
package mypack;
import pack.A;
class B
{
    public static void main(String a[])
    {
        A obj = new A();
        obj.msg();
    }
}
```

Output: Hello

3. Using fully qualified name

- If you use fully qualified name then only declared class of this package will be accessible.
- Now there is no need to import. But you need to use fully qualified name every time when you are accessing the class or interface.

Example

```
//A.java
package pack;
public class A
{
    public void msg()
    {
        System.out.println("Hello");
    }
}

//B.java
package mypack;
class B
{
    public static void main(String a[])
    {
        pack.A obj = new pack.A(); //using fully qualified
        obj.msg();
    }
}
```

Output: Hello

Interfaces in Java

- An **interface in Java** is a blueprint of a class. It has static constants and abstract methods.
- The interface in Java is *a mechanism to achieve [abstraction](#)*.
- There can be only abstract methods in the Java interface, not method body.
- It is used to achieve abstraction and multiple [inheritance in Java](#).
- It cannot be instantiated just like the abstract class.

Interfaces in Java

- **How to declare an interface?**
- An interface is declared by using the interface keyword.
- It provides total abstraction; means all the methods in an interface are declared with the empty body, and all the fields are public, static and final by default.
- A class that implements an interface must implement all the methods declared in the interface.

Interfaces in Java

Syntax:

```
interface <interface_name>
{
    // declare constant fields
    // declare methods that are abstract by default.
}
```

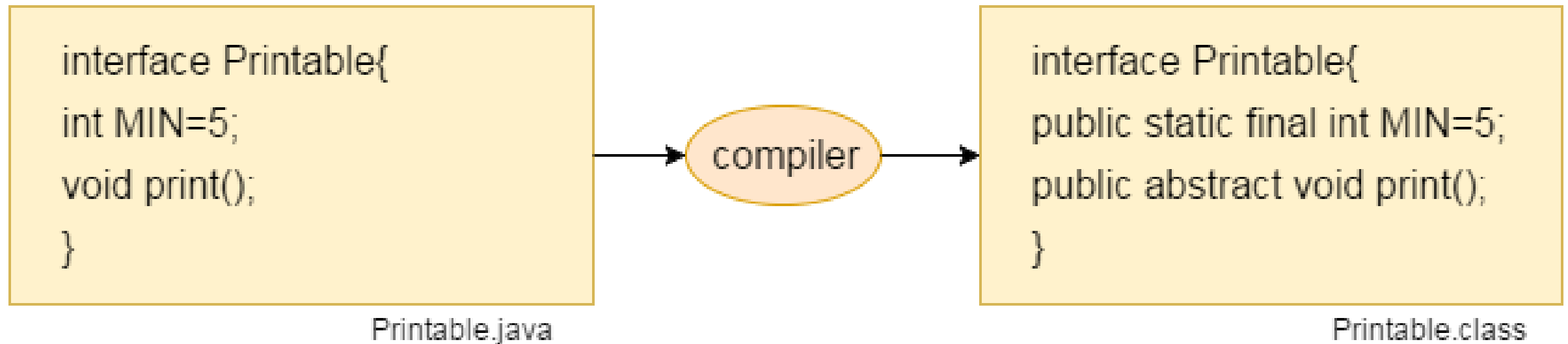
Example

```
interface printable
{
    void print();
}
```

Interfaces in Java

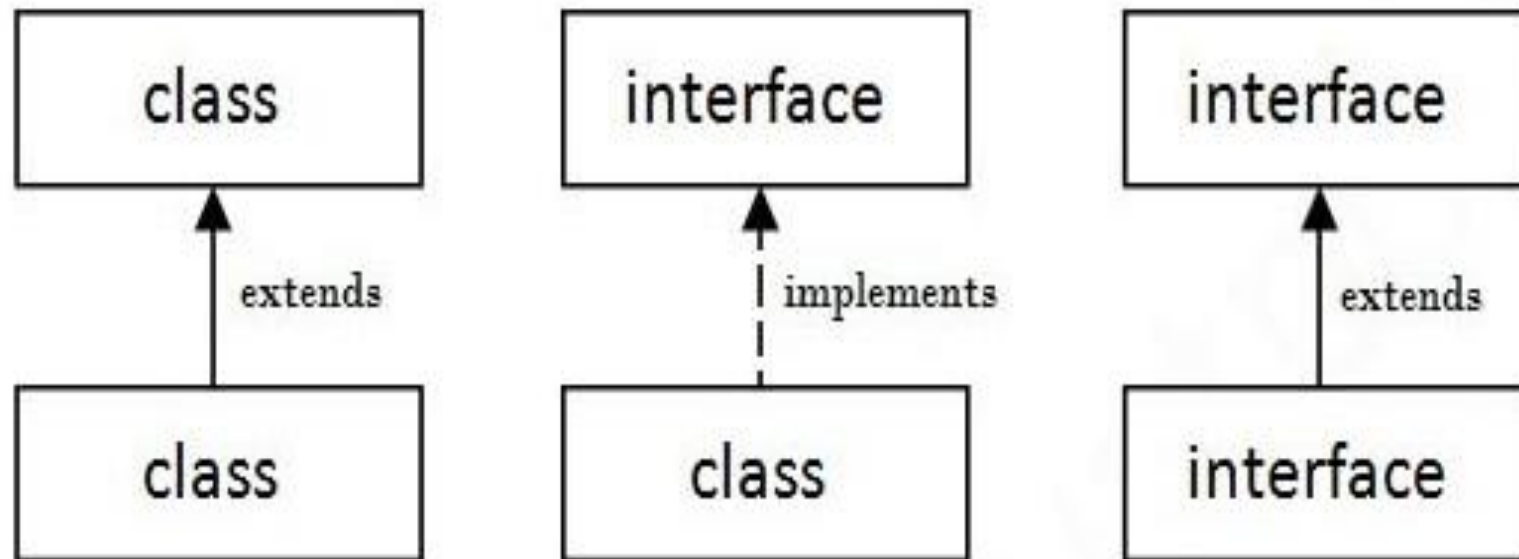
- **Internal addition by the compiler**
- The Java compiler adds public and abstract keywords before the interface method.
- Moreover, it adds public, static and final keywords before data members.
- In other words, Interface fields are public, static and final by default, and the methods are public and abstract.

Internal addition by the compiler



Interfaces in Java

- ***The relationship between classes and interfaces***
- As shown in the figure given below, a class extends another class, an interface extends another interface, but a **class implements an interface**.

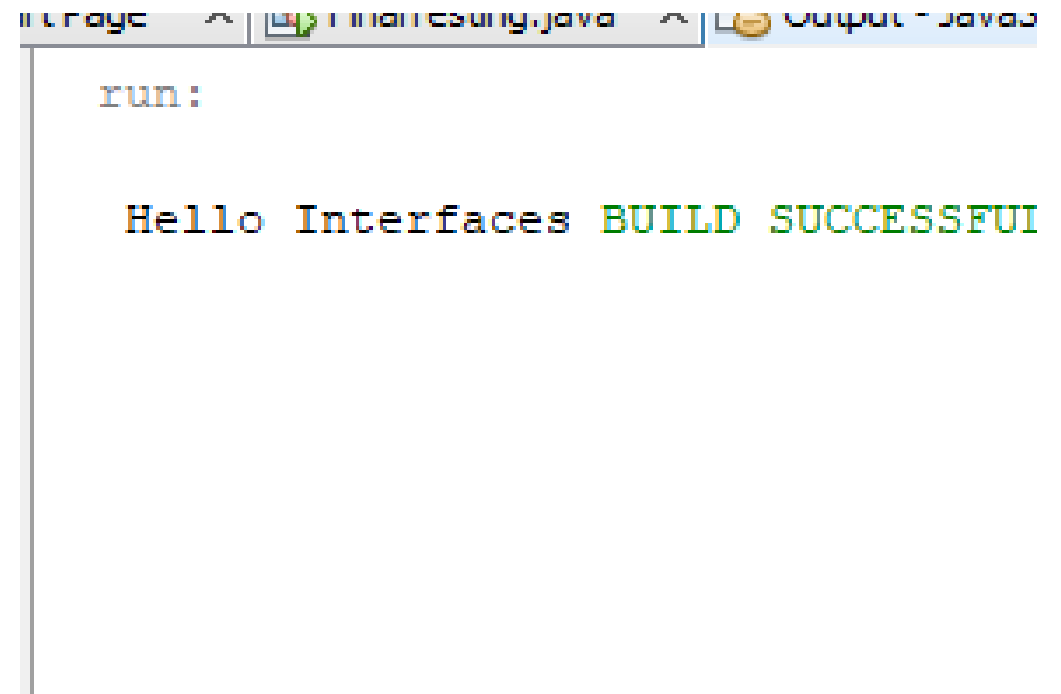


// Java Interface Example

- interface printable
- {
- void print();
- }
- class A6 implements printable
- {
- public void print()
- {
- System.out.println("Hello");
- }
- public static void main(String args[])
- {
- A6 obj = new A6();
- obj.print();
- }
- }

```
// Interface Example
package Interfaces2;
interface printable
{
    public void print();
}
class PrintingClass implements printable
{
    public void print()
    {
        System.out.print("\n Hello Interfaces ");
    }
}
class A6
{
    public static void main(String args[])
    {
        PrintingClass obj = new PrintingClass();
        obj.print();
    }
}
```

```
// Interface Example
package Interfaces2;
interface printable
{
    public void print();
}
class PrintingClass implements printable
{
    public void print()
    {
        System.out.print("\n Hello Interfaces ");
    }
}
class A6
{
    public static void main(String args[])
    {
        PrintingClass obj = new PrintingClass();
        obj.print();
    }
}
```

A screenshot of an IDE window with three tabs: 'Interface', 'Interfacesung.java', and 'Output - Java2'. The 'Interfacesung.java' tab is active, showing the same Java code as the first block. The 'Output - Java2' tab shows the program's execution result. The output is displayed in a monospaced font, with the text 'Hello Interfaces' in black, 'BUILD' in green, and 'SUCCESSFUL' in red.

```
run:

Hello Interfaces BUILD SUCCESSFUL
```

Notes on Interfaces:

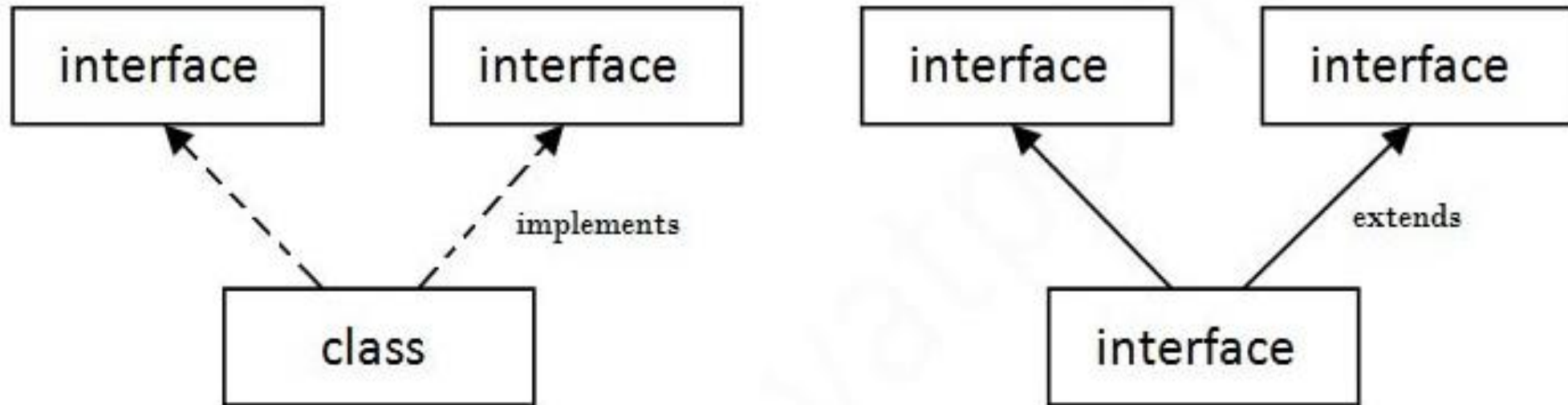
1. Like abstract classes, interfaces cannot be used to create objects
2. Interface methods do not have a body - the body is provided by the "implement" class
3. On implementation of an interface, you must override all of its methods
4. Interface methods are by default **abstract and public**
5. Interface attributes are by **default public, static and final**
6. An interface cannot contain a constructor

Why And When To Use Interfaces?

- **1) To achieve security –**
 - Hide certain details and only show the important details of an object (interface).
- **2) To Achieve Multiple Inheritance –**
 - Java does not support "multiple inheritance" (a class can only inherit from one superclass).
 - However, it can be achieved with interfaces, because the class can implement multiple interfaces.
 - Note: To implement multiple interfaces, separate them with a comma (see example below).

Multiple inheritance in Java by interface

- If a class implements multiple interfaces, or an interface extends multiple interfaces, it is known as multiple inheritance.



Multiple Inheritance in Java


```
// Multiple inheritance using interfaces
package Interfaces;
interface Printable{
    void print();
}
interface Showable{
    void show();
}
class PrintClass implements Printable,Showable {
    public void print()
    {System.out.println("Hello Printable");}
    public void show()
    {System.out.println("Welcome Showable");}
}
public class A7 {
    public static void main(String args[]){
        PrintClass obj = new PrintClass();
        obj.print();
        obj.show();
    }
}
```

```
// Multiple inheritance using interfaces
package Interfaces;
interface Printable{
    void print();
}
interface Showable{
    void show();
}
class PrintClass implements Printable,Showable {
    public void print()
    {System.out.println("Hello Printable");}
    public void show()
    {System.out.println("Welcome Showable");}
}
public class A7 {
    public static void main(String args[]){
        PrintClass obj = new PrintClass();
        obj.print();
        obj.show();
    }
}
```

```
run:
Hello Printable
Welcome Showable
BUILD SUCCESSFUL (total time: 0 secor
```

// Multiple inheritance using interfaces

```
// Multiple inheritance using interfaces
package Interfaces;
interface Printable{
    void print();
}
interface Showable{
    void show();
}
class PrintClass implements
Printable,Showable {
    public void print()
    {System.out.println("Hello Printable");}
```

```
        public void show()
        {System.out.println("Welcome
Showable");}
    }
    public class A7 {
        public static void main(String args[]){
            PrintClass obj = new PrintClass();
            obj.print();
            obj.show();
        }
    }
```

Multiple inheritance using interfaces

- Multiple inheritance is not supported in the case of class because of ambiguity.
- However, it is supported in case of an interface because there is no ambiguity.
- It is because implementation of methods of the interface is provided by the implementation class.

Abstract class	Interface
1) Abstract class can have abstract and non-abstract methods.	Interface can have only abstract methods. Since Java 8, it can have default and static methods also.
2) Abstract class doesn't support multiple inheritance.	Interface supports multiple inheritance.
3) Abstract class can have final, non-final, static and non-static variables.	Interface has only static and final variables.
4) Abstract class can provide the implementation of interface.	Interface can't provide the implementation of abstract class.

Abstract class	Interface
5) The abstract keyword is used to declare abstract class.	The interface keyword is used to declare interface.
6) An abstract class can extend another Java class and implement multiple Java interfaces.	An interface can extend another Java interface only.
7) An abstract class can be extended using keyword "extends".	An interface can be implemented using keyword "implements".
8) A Java abstract class can have class members like private, protected, etc.	Members of a Java interface are public by default.

Java 8 Interface Improvement

- Since [Java 8](#), interface can have default and static methods

Java 8 Interface Improvement

- Since Java 8, we can have method body in interface.
- But we need to make it default method.
- Eg.

interface Drawable

```
{  
    void draw();  
    default void msg()  
    {System.out.println("default method");}  
}
```



```
// Default methods with body in interfaces
package Interfaces2;
interface Drawable
{
    void draw();
    default void msg()
    {System.out.println("default method");}
}
class Rectangle implements Drawable{
    public void draw()
    {System.out.println("drawing rectangle");}
}
class TestInterfaceDefault{
    public static void main(String args[]){
        Drawable d=new Rectangle();
        d.draw();
        d.msg();
    }
}
```

```
// Default methods with body in interfaces
package Interfaces2;
interface Drawable
{
    void draw();
    default void msg()
    {System.out.println("default method");}
}
class Rectangle implements Drawable{
    public void draw()
    {System.out.println("drawing rectangle");}
}
class TestInterfaceDefault{
    public static void main(String args[]){
        Drawable d=new Rectangle();
        d.draw();
        d.msg();
    }
}
```

```
run:
drawing rectangle
default method
BUILD SUCCESSFUL (total time: 0
```

// Multiple inheritance using interfaces

```
// Multiple inheritance using interfaces
package Interfaces;
interface Printable{
    void print();
}
interface Showable{
    void show();
}
class PrintClass implements
Printable,Showable {
    public void print()
    {System.out.println("Hello Printable");}
```

```
        public void show()
        {System.out.println("Welcome
Showable");}
    }
    public class A7 {
        public static void main(String args[]){
            PrintClass obj = new PrintClass();
            obj.print();
            obj.show();
        }
    }
```

Java 8 Static Method in Interface

- Since Java 8, we can have static method in interface. Let's see an example:

```
interface Drawable
{
    void draw();
    static int cube(int x){return x*x*x;}
}
class Rectangle implements Drawable
{
    public void draw(){System.out.println("drawing rectangle");}
}
```

// Static methods in interfaces

package Interfaces;

interface Drawable

{

void draw();

static int cube(int x){return x*x*x;}

}

class Rectangle implements Drawable

{

public void draw(){System.out.println("drawing rectangle");}

}

class TestInterfaceStatic

{

public static void main(String args[]){

Drawable d=new Rectangle();

d.draw();

System.out.println(Drawable.cube(3));

}

}

// Static methods in interfaces

package Interfaces;

interface Drawable

{

void draw();

static int cube(int x){return x*x*x;}

}

class Rectangle implements Drawable

{

public void draw(){System.out.println("drawing rectangle");}

}

class TestInterfaceStatic

{

public static void main(String args[]){

Drawable d=new Rectangle();

d.draw();

System.out.println(Drawable.cube(3));

}

}

run:

drawing rectangle

27

BUILD SUCCESSFUL (total time: 0

// Satic methods in interfaces

// Satic methods in interfaces

```
package Interfaces;
interface Drawable{
void draw();
static int cube(int x){return x*x*x;}
}
class Rectangle implements Drawable
{
    public void
draw(){System.out.println("drawing
rectangle");}
}
```

```
class TestInterfaceStatic
{
    public static void main(String args[]){
        Drawable d=new Rectangle();
        d.draw();

System.out.println(Drawable.cube(3));
    }
}
```